

Anomaly detection for time series infrastructure metric data

Ing. Stefan-Lucian Bucur
POLITEHNICA University of Bucharest
Bucharest, Romania
slbucur@gmail.com

Prof. Dr. Ing. Florica Moldoveanu
POLITEHNICA University of Bucharest
Bucharest, Romania
florica.moldoveanu@cs.pub.ro

Abstract — Time series metric data is used by monitoring systems to analyze the performance of hardware and software from a data center. This data can be easily displayed using line graphs and, as such, can be manually checked by operators for anomalies. Manual reviews have some major disadvantages, since they are time consuming and error prone. In this paper, we describe our approach for automatically detecting outliers in metric data, which have some unique properties like high seasonality and extreme seasonal spikes. We use the “Prophet” library to create a model of the time series data, which is then automatically analyzed with our own distance formula for detecting anomalies in the time series data. A part of our metrics data have a propensity for extreme recurrent spikes, for example network usage during daily backups that is millions of times higher than the average. To handle such special cases we have also implemented a seasonal spike detector.

Keywords— *time series, infrastructure metrics, anomaly detection*

I. INTRODUCTION

The STAID division within Cegeka offers a complex monitoring service for customers that have devices in the Cegeka cloud, but also in their own private data centers.

The monitoring service has two main areas:

- **Performance data** – this represents data about servers that can be plotted on a graph. These data can range from basic, for example memory utilization, to complex, for example custom SQL scripts that monitor database usage.
- **Alerting** – the monitoring system sends alerts based on preset criteria. At the most basic level, alerts need to be generated when a server is no longer available, but many other types of alerts are also available.

The performance data is actually time series data: lists of tuples with a timestamp and a numeric value, most of the time a floating-point number. This is represented graphically as a plot with one or more lines.

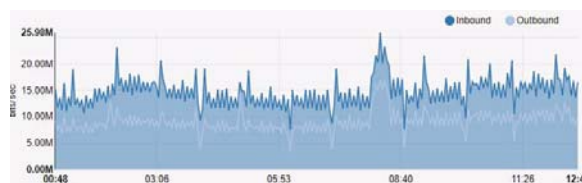


Fig. 1. Network throughput graph represented by inbound and outbound traffic (inbound – dark blue, outbound – light blue)

The networking team uses a subset of the networking graphs to generate monthly availability reports for our customers. The team also manually analyzes the graphs to spot

anomalies like sudden spikes that can be the result of security breaches or incorrect device-level configurations.

The number of devices that Cegeka manages has more than tripled in the last few years, which makes manually inspecting the graphs time consuming and error prone. Our task was to automate the process and as a result to identify more anomalies, and to free up our engineers for other assignments.

The approach we had was to initially implement a proof of concept (POC) using a statistical model, and to use this as a starting point for implementing more complex deep learning models in the future.

II. STATE OF THE ART

To implement automatic anomaly detection, we have analyzed the current state of the art, and found that there are two main approaches: statistical models and deep learning.

1) Statistical models

In this approach, a model is defined for the data, composed of one or more time dependent functions. The functions are usually defined by multiple static or dynamic parameters, and a process called “fitting” or “training” is used to find the appropriate values of the parameters that would match the most points for a given dataset.

Because the models need to be manually defined, they usually cannot model very complex datasets, like stock market data, but are appropriate for simpler ones. Usually statistical models are very fast to train and work using single time series metric as an input, therefore a model needs to be trained for each metric.

Some examples would be:

- STL decomposition - Seasonal Trend decomposition using Loess [1]
- ARIMA – Auto Regressive Integrated Moving Average [2]
- ETS decomposition – Error, Trend, Seasonality [3]

2) Deep learning models

These models use a neural network that is essentially comprised of multiple nonlinear models, one for each neuron in the network [4].

Because we can use large number of neurons, a deep neural network can simulate very complex models [4], [5].

Deep learning models can take a very large time to train compared to the statistical models, and as such are more useful when having multiple metrics as inputs [6].

III. DATA DESCRIPTION

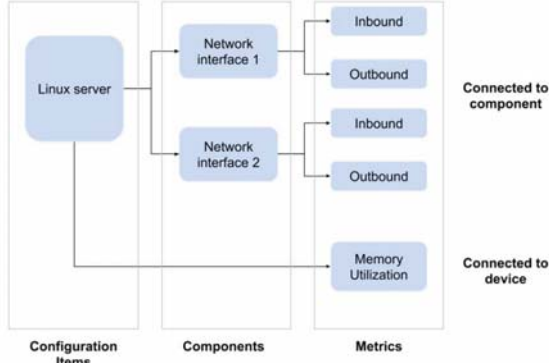


Fig. 2. Example monitoring data for a Linux server

The anomaly detection project focuses on device metric data coming from our main monitoring tool, Zenoss [7], which currently monitors approximately 20.000 devices.

A “device” is a functional unit required to run a customer service. It can be a hardware item (HI), for example a physical server or a network switch, or a software item (SI), such as a virtual server or a database. In services management terminology, a device is referred to as a configuration item (CI) or an asset [8].

A device also has a number of associated “components”, for instance, a server can have multiple file systems and a switch usually has multiple network interfaces.

The metric data is usually associated to components, for example files system usage is associated with the device file system component. A metric can also be associated directly to a device, in two instances: it refers the device itself, such as system uptime, or it is an aggregate of all the components of a device, for example total file system usage.

The relations between metrics and devices or components can be retrieved in an indirect way, by using graph definitions. A graph definition is used to plot metric data in graphs. For example, a network interface on a Linux server has multiple graphs: throughput, packets and errors. The throughput definition consists of two line graphs, one for the packets that are being sent successfully from the device (Outbound) and one for the packets that are being received successfully by device (Inbound).

TABLE I. DATA ITEMS AND THEIR APPROXIMATE COUNT

Data Item	Count
Device	20.000
Components	1.300.000
Graph definitions	1.500.000
Metrics	3.000.000

From our initial analysis, we have observed that the number of components is 100 times greater than the number of devices, and that, on average, a component has two metrics.

Because of the large number of metrics, the metric data is stored separately from the configuration data, in an open source time series database called OpenTSDB [9].

This database stores each individual metric separately, which has two consequences:

- The data is easily compressed, since metric data is usually periodic
- The data can be easily distributed to multiple databases

In OpenTSDB, a time series data point consists of:

- A metric name.
- A UNIX timestamp (seconds or milliseconds since Epoch).
- A value – usually a 64 bit integer or single-precision floating point value
- A set of tags (key-value pairs) that describe the time series the point belongs to

Because of these properties, it is easy to query a single metric from this OpenTSDB but harder to do complex analytics on a database level.

A very useful feature [10] of the database is the ability to down sample (aggregate) metrics. Most our metrics are sampled at a 3 minute interval, but we can easily aggregate to any interval above 3 minutes that we need.

TABLE II. EXAMPLE METRIC DATA DOWN SAMPLING

Original data		Aggregated data			
Time	Value	5 Minute Average	5 Minute Minimum	5 Minute Maximum	
2018-01-01 00:00:00	10	11	10	12	
2018-01-01 00:03:00	12	10	10	10	
2018-01-01 00:06:00	10	12	12	12	
2018-01-01 00:09:00	10	13.5	13	14	
2018-01-01 00:12:00	12	14.5	13	16	
2018-01-01 00:15:00	13	12	12	12	
2018-01-01 00:18:00	14	10	10	10	
2018-01-01 00:21:00	16				
2018-01-01 00:24:00	13				
2018-01-01 00:27:00	12				
2018-01-01 00:30:00	10				
Time	15 Minute Average	15 Minute Minimum	15 Minute Maximum		
2018-01-01 00:00:00	10.8	10	12		
2018-01-01 00:15:00	13.6	12	16		

Regarding data retention and availability, we have started migrating device monitoring to the sixth version of Zenoss since February, with the bulk of our devices being migrated since July, so for most devices we have at least 6 months of metric data.

Because of the large number of metrics, we only retain the data for 5 months, after which it is archived to a data warehouse, down sampled to 1 hour.

IV. TIME SERIES ANOMALY DEFINITION

Anomalies can be defined in two ways:

- Manually, when an engineer labels a timestamp and its value as being an outlier
- Statistically, when a value is statistically different than others

Currently we do not have manually labeled data, so our goal is at first to implement a statistical model for anomaly detection, after which engineers can label if the model properly identified anomalies. Using the labeled data, we can then improve our model.

In the interface graph represented in figure 3, it is easily observable that on October 25, October 1 and on November 5 the data has some abnormal spikes that exceed the daily average by a factor of 4.

Not all spikes are anomalies, though. Since most our customers are active only in Europe, our metric data manifests a high degree of daily and weekly seasonality. Devices are used more during the day and during the working days of the week, and less in the night and in the weekends.

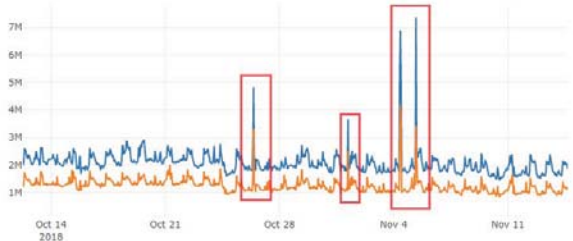


Fig. 3. Anomalies in a network interface throughput graph

For example, in the following dataset shown in figure 4, there is a daily spike at 10 AM, but this is not an anomaly, since it is repeated every day.



Fig. 4. Time series with weekly seasonality (above) and daily seasonality (below)

The same dataset also manifests weekly seasonality; it can be observed that the values are lower 2 out of 7 days, which are the weekends.

V. PROPHET FORECASTING MODEL

For the anomaly detection proof of concept, we have initially tried to use the simpler available models: ARIMA and ETS decomposition.

None of these two was appropriate for all of our datasets, because they were easily influenced by the variance of the values and detected either too many or not enough anomalies.

A. Prophet library

The model that we decided to use is based on the **Prophet** [11] library developed by Facebook research [12].

The library uses Bayesian inference to fit a time series dataset to an additive model, described in the following formula:

$$y(t) = g(t) + s(t) + h(t) + e(t)$$

In this formula:

- $y(t)$ – model output function
- $g(t)$ – trend of the dataset
- $s(t)$ – seasonality of the dataset
- $h(t)$ – holidays
- $e(t)$ – error

For our dataset, we focus mainly on the trend and the seasonality.

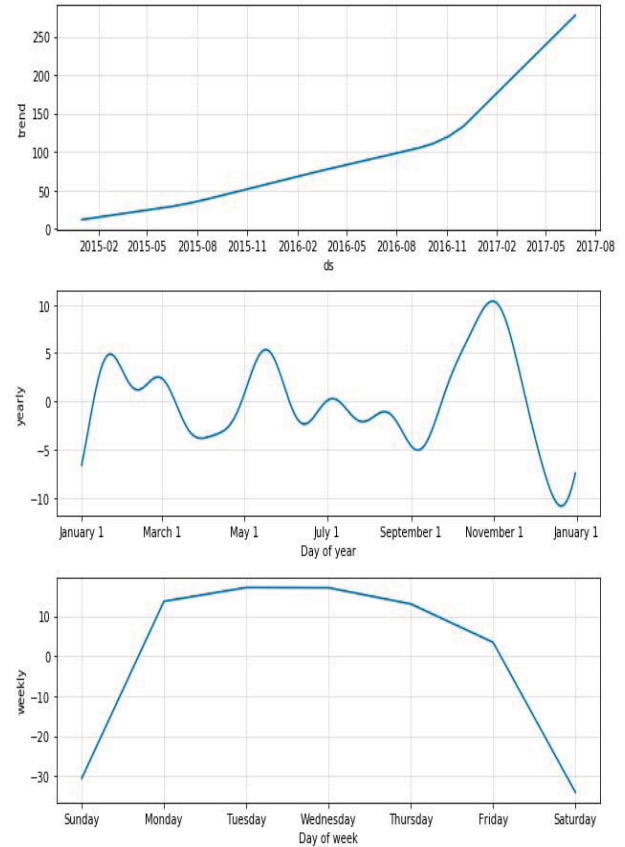


Fig. 5. Plots of the trend and seasonal components of a prophet model

1) Trend

The trend is the general direction of dataset. In the prophet library, it can be of two types:

- Linear – the output is represented by a line
 - Prophet uses a **Piecewise Linear Model** which is a series of linear models, one model per predefined period (daily by default)
- Logistic – the output is represented by a curve
 - Prophet uses a **Nonlinear, Saturating Growth** that can model exponential growth

2) Seasonality

Prophet can cover three types of seasonality:

- Daily – repeating values during a day
- Weekly – repeating values during a week
- Yearly – repeating values during a year

After the seasonality component is calculated, it will result in a list of values that depend on the time of the day (hour, day of the week and day of the year).

There are two ways seasonality can be added to the final model function:

- Additive – the seasonality is added to the trend component
- Multiplicative – the seasonality is multiplied with the trend component

For our use case, we used the additive method.

a) Bayesian inference using Stan

Both **trend** and **seasonality** are modeled using Bayesian inference, a method of statistical inference in which Bayes' theorem is used to update the probability for a hypothesis as more evidence or information becomes available.

This type of statistical analysis is done automatically using the Stan computer programming language, in which an inference model is specified and the library does the modeling.

```
if (trend_indicator == 0) {
  y ~ normal(
    linear_trend(k, m, delta, t, A, t_change)
    .* (1 + X * (beta .* s_m))
    + X * (beta .* s_a),
    sigma_obs
  );
} else if (trend_indicator == 1) {
  y ~ normal(
    logistic_trend(k, m, delta, t, cap, A, t_change, S)
    .* (1 + X * (beta .* s_m))
    + X * (beta .* s_a),
    sigma_obs
  );
}
```

Fig. 6. Sample of the Stan model used in the prophet library [13]

In figure 8, we can see a sample of the Stan model used by prophet. The program depends on multiple user-supplied inputs. We can see in this case that the **trend_indicator** variable is used to determine how the trend function is calculated: linearly or logistically.

VI. IMPLEMENTATION

A. Anomaly visualization interface

We developed an interface to visualize the anomalies in our graphs. The graphs needed to look visually similar to the ones from the Zenoss backend.

In this interface, a user can select:

- The device (configuration Item)
- The component type (ex. File system, interface)
- The component (Ex. File system /home, Network interface eth0)
- The graph (Ex. File system utilization)
- The time range (by default the last 3 days)
- The down sample (aggregation interval, ex. 3 minute, 5 minute average, 1 hour average)

As a result, a graph is displayed with the following information:

- A line graph for each metric
- For each anomaly, a bubble (circle) proportional to the confidence of the anomaly

Since some points are more distanced from the normal, they are more likely anomalies. The confidence of the anomaly ranges from zero to one. Points with zero percent confidence are not anomalies, while points with a confidence of 100 percent are definitely anomalies.

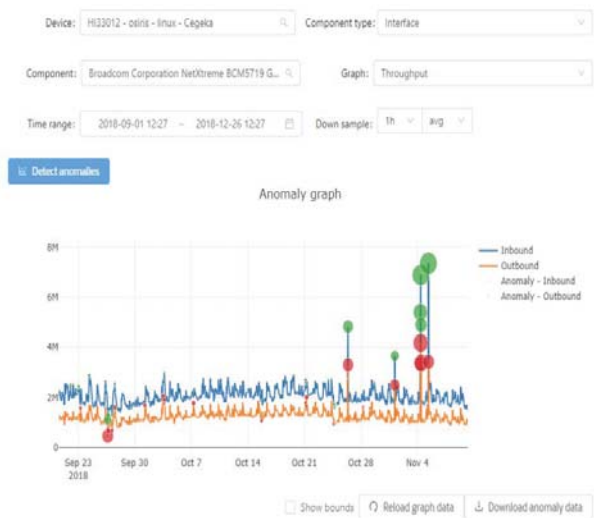


Fig. 7. Anomaly visualization interface

B. Anomaly detection model

Using the Prophet library, we can generate a model of our time series data that does not include very high variations.

After training the prophet model with the metric dataset, we use the library and the model to generate a table [14] with the following columns:

- Timestamp – one for each data point in the original dataset
- Predicted value (y_{hat}) – the value that the model predicts
- Model upper and lower bounds (y_{hat_lower} and y_{hat_upper}) – the uncertainty interval of the model, the predicted value will usually be the average of these values

Based on the lower and upper bounds, we calculate the anomaly likelihood of a given point using the following formula:

$$A_l(t) = \frac{\min(\text{distance}(M_u(t), D(t)), \text{distance}(M_l(t), D(t)))}{\text{factor} \cdot (M_u(t) - M_l(t))}$$

In this formula:

- $A_l(t)$ – the anomaly likelihood of a timestamp
 - This will be a value between 0 and 1:
 - 0 – not an anomaly
 - 1 – definitely an anomaly
- $D(t)$ – the value of the timestamp in the original dataset
- $M_l(t)$ – the Model lower bound
- $M_u(t)$ – the Model upper bound
- Factor – number multiplied with the difference between the upper and lower bounds
 - For example, a factor of 2 means that if the distance between a point and its closest bound is double the distance between the bounds, the anomaly likelihood will be 1 (or 100% percent, full anomaly)

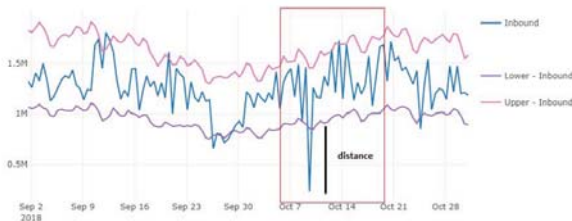


Fig. 8. Components used in the anomaly likelihood calculation: Inbound (original dataset), Lower – Inbound (lower bound), Upper – Inbound (upper bound)

C. Special cases

This model works well for time series data that do not have a very high variation that is considered normal.

One of the cases we immediately had problems with was for devices that have very steep daily spikes. This is usually the case with a device where the network interface is rarely used, but the device has daily backups.

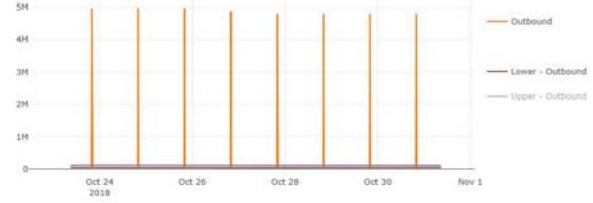


Fig. 9. Graph showing exceedingly high daily spikes

In the example above, the daily backups happen every day at 8 PM, which causes a 1000 X increase from the other values that appear during the day. The Prophet library cannot model this high variation since it is statistically extremely different from the rest of the dataset.

For these particular cases, we implemented a seasonal spike detector that eliminates the spikes from the anomaly set. The seasonal spike detector does the following steps:

- Find all the peaks that are at least 1 hour away from each other
- Calculate the time intervals between the peaks
- Find the most common interval
- If this distance appears more than 3 times, set the values of the spikes at the interval median before fitting the model using the prophet library

Using this approach, we can eliminate the set of false anomalies that would normally be identified in situations like those that are displayed in figure 11.

VII. CONCLUSION AND FUTURE IMPROVEMENTS

Using the Prophet library and the seasonal peak detector produced good initial results.

The anomaly likelihood formula developed by us has the advantage of being decoupled from time series model. Because of this, it can easily be used with other forecasting methods, such as ARIMA [2].

The sensitivity of the formula can also be easily adjusted using the ‘factor’ parameter, which can be changed directly in the visualization in order to eliminate noise.

Although our current approach is a useful starting point it has some inherent problems.

The first problem is the raw performance:

- For small datasets (under 1000 samples), model fitting and predicting the outliers takes between 4 and 10 seconds on a virtual machine with one core
- It would be unfeasible to use this approach to check for anomalies in all our metrics (3 million) every 3 minutes - as this would require thousands of machines

Another issue is the lack of additive training:

- Once a model is fitted with a dataset, it cannot be further trained with additional data points
- Since the training period is the largest, this feature could improve inference times dramatically

Having one model per metric also has some drawbacks:

- With this approach, we cannot use multiple metrics as an input for anomaly detection

- By using multiple metrics, daily high spikes could be easily identified as non-anomalies, since CPU and disk usage are also high during that peak

In our initial exploration, we also trained deep neural network models using a LSTM network (generally recommended for time series data) [5]. We found this approach unfeasible for one model per metric, since the training period could easily exceed 10 minutes, even on custom hardware. A deep neural network can model highly complex behaviors, therefore our next action point for this project is to use a neural network that has multiple metrics as inputs. This should decrease the model training time and increase the prediction accuracy.

We also plan to thoroughly explore the Prophet library internals, considering it uses an open source BSD license, and find ways to increase performance and accuracy for our specific use case.

REFERENCES

- [1] R. B. Cleveland, W. S. Cleveland, J. E. McRae, I. Terpenning – STL: A Seasonal-Trend Decomposition Procedure Based on Loes, 1990
- [2] R. J. Hyndman; G. Athanasopoulos. – Forecasting: principles and practice. Seasonal ARIMA models. p84-93, May 2018 ISBN 0987507117
- [3] R. J. Hyndman; G. Athanasopoulos. – Forecasting: principles and practice. Forecasting with ETS models. P50-51, May 2018 ISBN 0987507117
- [4] J. Gamboa. Deep Learning for Time-Series Analysis, Seminar on Collaborative Intelligence in the TU Kaiserslautern. January 2016
- [5] P. Malhotra, L. Vig, G. Shroff, P. Agarwal.; Long short term memory networks for anomaly detection in time series. In: European Symposium on Artificial Neural Networks. vol. 23, June 2015
- [6] N. Laptev, S. Smyl, S. Shanmugam. “Time-series Extreme Event Forecasting with Neural Networks at Uber”, June 2017
- [7] The Zenoss monitoring tool - http://wiki.zenoss.org/Main_Page, Accessed April 2019
- [8] J.M. Quigley, K.L. Robertson, “Configuration Management - Theory, Practice, and Application”, Chapter “ A Configuration Item and What It Implies”, p130-131, April 2015, ISBN 9781482229363
- [9] S. Benoit. “OpenTSDB: A Scalable, Distributed Time Series Database”, 2011
- [10] Downsampling - OpenTSDB 2.3 documentation
http://opentsdb.net/docs/build/html/user_guide/query/downsampling.html, Accessed April 2019
- [11] S. J. Taylor, B. Letham. “Forecasting at Scale”, December 2017
- [12] Facebook Research. Available: <https://research.fb.com/>. Accessed April 2019
- [13] B. Letham. “Facebook prophet”, Github repository,
<https://github.com/facebook/prophet/blob/master/python/stan/unix/prophet.stan>. Accessed April 2019
- [14] S. J. Taylor, B. Letham, “Facebook prophet start guide”
https://facebook.github.io/prophet/docs/quick_start.html. Accessed April 2019